Ders 3 Çalışma Özeti

Ders 3 Çalışma Özeti

Genel Konular

- Dersin kaynak kitapları
- İyi bir algoritma tasarımının hedefleri
- Basit sıralama algoritmaları (sorting algorithms):
  - Kabarcık sıralaması (Bubble Sort) ve erken çıkışlı iyileştirilmiş hâli
  - Seçmeli sıralama (Selection Sort)
  - Araya eklemeli sıralama (Insertion Sort)
  - Shell sıralaması (Shell Sort)
- Karmaşıklık analizi: en iyi durum (best case), en kötü durum (worst case), ortalama durum (average case)
- Karmaşıklık sınıflarının çalışma süresine etkisi
- Yerinde sıralama (in-place sorting) ve kararlı sıralama (stable sorting) kavramları

Hocanın Özellikle Vurguladığı Kısımlar

- Bir algoritma tasarlanırken önce küçük bir örnek veri üzerinde denenmeli. Böylece uç durumlar ve hatalar daha kolay görülür.
- İlk tasarım her zaman en iyisi olmayabilir. Önce doğru çalışan kaba bir tasarım yapılır, sonra adım adım iyileştirilir.
- İş bitmiş olmasına rağmen algoritmanın çalışmaya devam etmesi çok sık yapılan bir hatadır. Devam etmeye gerek olup olmadığı kontrol edilmelidir.
- Adım sayısı belli olmayan bir döngü için `for` kullanıp `break` ile kırmak yerine `while` / `do-while` kullanılmalıdır.
- Yer değiştirme (swap) de bir maliyettir. Gereksiz yer değiştirmelerden kaçınılmalıdır.
- Karmaşıklık, makineden bağımsız olarak giriş boyutu n'e göre ifade edilir. "Şu bilgisayarda 10 dakika sürdü" bilgisi karşılaştırma için anlamlı değildir.
- Gerçek uygulamalarda çoğunlukla ortalama durum önemlidir. Ancak en kötü durumun ne zaman dikkate alınacağı problemin ihtiyacına göre kararlaştırılır.
- Küçük dizilerde en çok tercih edilen n^2 algoritması Insertion Sort'tur.
- Kararlılık (stability) kavramı pratikte önemlidir. Birden fazla ölçüte göre sıralama yaparken algoritma seçimi buna göre yapılmalıdır.

Kısa Tekrar Notları

- **Kaynak kitaplar:**
  - Algorithms, 4th Edition (Robert Sedgewick, Kevin Wayne): en çok kullanılan kaynak
  - Introduction to the Design & Analysis of Algorithms (Anany Levitin): ikinci kaynak
  - The Algorithm Design Manual (Steven S. Skiena): yan kaynak
- **Bubble Sort:** Ardışık iki eleman karşılaştırılır, büyük olan sağa geçer. Her turun sonunda bakılan bölgenin en büyük elemanı en sona yerleşir.
- **Selection Sort:** Her turda bakılan bölgenin en küçük elemanı bulunur. Bu eleman bölgenin ilk elemanıyla **bir kez** yer değiştirir.
- **Insertion Sort:** Her eleman, kendinden önceki sıralı kısımda yerini bulana kadar geriye taşınır. Yerini bulunca durur.
- **Shell Sort:** Insertion Sort'un adım büyüklüğü (step size / gap) ile yapılan hâlidir. Sondaki küçük elemanlar büyük adımlarla öne taşınır. Adım her turda yarıya indirilir. Son tur

klasik Insertion Sort'tur.
- **Karmaşıklık özeti:**

Algoritma	En iyi durum	En kötü durum	Kararlı mı?	Yerinde mi?
Bubble Sort (erken çıkışlı)	n ($n-1$ karşılaştırma)	n^2	Evet	Evet
Selection Sort	n^2	n^2	Hayır	Evet
Insertion Sort	n ($n-1$ karşılaştırma)	n^2	Evet	Evet
Shell Sort	-	adım dizisine bağlı	-	Evet
Merge Sort	$n \log n$	$n \log n$	-	Hayır (yedek dizi kullanır)
Quick Sort	-	n^2	Hayır	Evet

- $\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} = \frac{n^2-n}{2} \approx n^2$
- **Yerinde sıralama:** Yedek dizi kullanmadan, dizinin kendi üzerinde yapılan sıralama.
- **Kararlı sıralama:** Eşit anahtarlı elemanların önceki sıralamadaki göreli sırası bozulmaz.

Detaylı Açıklamalar

Dersin Kaynak Kitapları

Hoca derste kullandığı kitapları tahtaya yazdı (ekrandan, 00:04:25):

1. *Algorithms, 4th Edition*, Robert Sedgewick ve Kevin Wayne. En çok kullandığı kitap.
2. *Introduction to the Design & Analysis of Algorithms*, Anany Levitin. İkinci en çok kullandığı kitap.
3. *The Algorithm Design Manual*. Yan kaynak. Hocanın hatırladığına göre e-kitap sürümü de var.

- İnternette çok fazla kontrolsüz kaynak vardır. Bazıları yanlış ya da verimsiz olabilir.
- Bu yüzden güvenilir kaynaklardan öğrenmek önemlidir. Kitaplar kendini geliştirmek için de tavsiye edildi.

İyi Bir Algoritmanın Hedefleri

Hoca bir algoritma tasarlarken beş hedef olduğunu hatırlattı:

1. **Doğruluk:** Algoritma her koşulda ve uç noktalarda doğru çalışmalı.
2. **Verimlilik:** Hem zamanı hem yeri iyi kullanmalı. Aynı amaçla kullanılan birden fazla algoritma varsa, problemin şartlarına en uygun olan seçilmeli.
3. **Kodlama kolaylığı:** Tasarım rahat kodlanabilir olmalı.
4. **Sadelik:** "if kullanmayayım" diye karmaşık hileler yapmak istenen bir şey değildir. Çok ciddi zaman veya yer kazancı yoksa sade ve temiz bir algoritma yazılmalıdır.
5. **Genellik:** Algoritma yalnız belli türdeki veriler (örneğin yalnız çift sayılar, yalnız 100'e kadar sayılar) için değil, genel olarak çalışmalı.

Bubble Sort (Kabarcık Sıralaması)

Çalışma mantığı:

- Ardışık iki eleman karşılaştırılır. $a[j] > a[j+1]$ ise yer değiştirilir.
- Bir turun sonunda bakılan bölgenin en büyük elemanı en sona taşınmış olur.
- Bir sonraki turda o eleman artık bakılan bölgeye dahil edilmez.
- Diğer elemanlar toplu olarak yerleşmez. Her turda yalnız bir eleman kesin yerine yerleşir.

Örnek dizi (ekrandan): `6 2 11 7 5`

Tur	Turun sonundaki dizi	Yerleşen eleman
1	`2 6 7 5 11`	11
2	`2 6 5 7 11`	7
3	`2 5 6 7 11`	6
4	`2 5 6 7 11`	5 ve 2

En kötü durum: Dizi ters sıralıysa (örneğin `11 9 7 5 2`) her turda yer değiştirme olur. 5 eleman için 4 tur gerekir. Genel olarak n eleman için $n-1$ tur yapılır.

İlk tasarım (ekrandaki akış diyagramından):

- Dış döngü: $i = 1$'den $n-1$'e kadar.
- İç döngü: $j = 0$'dan $n-i-1$'e kadar.
- $i = 1$ için en son $a[n-2]$ ile $a[n-1]$ karşılaştırılır.
- $i = n-1$ için $n-i-1 = 0$ olur. Yalnız $a[0]$ ile $a[1]$ karşılaştırılır.

Hoca bu sınırları ters sıralı bir dizi üzerinde adım adım kontrol etti. Amaç, her iki döngünün de tüm şartları sağladığını görmekti.

İlk tasarımın sorunu: Program doğru çalışır ama verimli değildir. Dizi zaten sıralıysa (örneğin `2 3 5 9 11`) birinci turda hiç yer değiştirme olmaz. Buna rağmen algoritma körlemesine $n-1$ tur dönmeye devam eder.

İyileştirme: Yer değiştirme olmuyorsa dizi sıralıdır, devam etmeye gerek yoktur.

- Döngü artık adım sayısı belli olmayan bir döngüdür. Bu yüzden `do-while` kullanılır.
- Her turun başında `swapped = 0` yapılır. Bir yer değiştirme olunca `swapped = 1` olur.
- Döngü şartı: `swapped && i < n`.
- Öğrenci, `swapped` dışında $i < n$ şartının da gerektiğini söyledi. Hoca bunu onayladı: dizi ters sıralıysa son turda hâlâ yer değiştirme olur.

Akış diyagramlarının C karşılığı (derste C kodu yazılmadı; akış diyagramından ajan tarafından çevrildi, derlenip `6 2 11 7 5` dizisiyle test edildi):

```c
void bubble_sort(int a[], int n) {
    int i = 1, j, swapped;
    do {
        swapped = 0;
        for (j = 0; j <= n - i - 1; j++)
            if (a[j] > a[j + 1]) {
                swap(&a[j], &a[j + 1]);
                swapped = 1;
            }
        i++;
    } while (swapped && i < n);
}
```

Selection Sort (Seçmeli Sıralama)

Bubble Sort'ta bir eleman, örneğin en büyük eleman, sona taşınırken sürekli yer değiştirir. Selection Sort bu maliyeti azaltmak için tasarlanmıştır:

- Bir tur boyunca bakılan bölgenin en küçük elemanı bulunur.
- Bu eleman, bölgenin ilk elemanıyla **tek bir kez** yer değiştirir.

Bir öğrenci "en küçüğü bulur, en başa yapıştırır" dedi. Hoca, "yapıştırmanın" aslında **en baştaki elemanla yer değiştirmek** olduğunu vurguladı.

Örnek (ekrandan, 01:02:00): `2 7 4 9 3 1 8`

Adım (i)	En küçük eleman	Yer değiştiren	Sonuç
0	1 (5. gözde)	$2 \leftrightarrow 1$	`1 7 4 9 3 2 8`
1	2	$7 \leftrightarrow 2$	`1 2 4 9 3 7 8`

Adım (i)	En küçük eleman	Yer değiştiren	Sonuç
2	3	$4 \leftrightarrow 3$	1 2 3 9 4 7 8
3	4	$9 \leftrightarrow 4$	1 2 3 4 9 7 8
4	7	$9 \leftrightarrow 7$	1 2 3 4 7 9 8
5	8	$9 \leftrightarrow 8$	1 2 3 4 7 8 9

Akış diyagramı (ekrandan):

- $i = 0$'dan $n - 2$'ye kadar:
  - `min = a[i]`, `adr = i`
  - $j = i + 1$'den $n - 1$'e kadar: `a[j] < min` ise `min = a[j]`, `adr = j`
  - `a[i] ↔ a[adr]`

C karşılığı (akış diyagramından ajan tarafından çevrildi, derlenip test edildi):

```c
void selection_sort(int a[], int n) {
    int i, j, min, adr;
    for (i = 0; i <= n - 2; i++) {
        min = a[i];
        adr = i;
        for (j = i + 1; j <= n - 1; j++)
            if (a[j] < min) {
                min = a[j];
                adr = j;
            }
        swap(&a[i], &a[adr]);
    }
}
```

Avantaj ve dezavantaj:

- Avantajı: Yer değiştirme sayısını en aza indirir.
- Dezavantajı: Dizi sıralı olsa bile en küçük elemanı aramaya devam eder. Bakmadan bölgenin en küçüğünü bilemez. Bu yüzden istenen yerde durma şansı yoktur.
- Selection Sort'ta dizinin küçükten büyüğe ya da büyükten küçüğe sıralı olması arasında fark yoktur. En iyi durum en kötü duruma eşittir.
- Gerçek veriler genelde sıralı gelmez. Ortalama durumda Selection Sort, Bubble Sort'tan daha iyidir.

Insertion Sort (Araya Eklemeli Sıralama)

n^2 karmaşıklıklı algoritmalar içinde en verimli olanıdır. Küçük dizileri sıralamak için tercih edilir.

- Quick Sort, $n \log n$ karmaşıklığı nedeniyle en çok kullanılan sıralama algoritmasıdır. İleriki haftalarda böl ve yönet (divide and conquer) algoritmalarıyla birlikte anlatılacak.
- Quick Sort özyinelemeli (recursive) bir algoritmadır. Küçük dizilerde sürekli fonksiyon çağırmak yavaşlığa neden olur.
- Bu yüzden Java, küçük dizi parçaları için Insertion Sort kullanır.
- 20-50 elemanlı küçük diziler için Insertion Sort verimli bir çözümdür.

Mantık: Eleman, kendinden öncekilerle karşılaştırılır. Yerini bulduğu anda durur ve yerleşir. Başa kadar gitmesi ya da her şeyi taraması gerekmez.

- Baş taraf kendi içinde sıralı olduğu için, eleman bir önceki elemandan büyükse daha öncekilere bakmaya gerek yoktur.
- Yarı sıralı dizilerde ve sıralı bir diziye arada yeni elemanlar eklendiği durumlarda çok avantajlıdır.

Örnek (ekrandan, 01:17:05): `3 2 8 5 9 1`

- 2, 3 ile karşılaştırılır. 3 bir kaydırılır, 2 başa yerleşir: `2 3 8 5 9 1`
- 8, 3'ten büyük. Geri kalanına bakılmaz.

- 5, 8'den küçük. 8 kaydırılır. 5, 3'ten büyük, durur: `2 3 5 8 9 1`
- 9 yerinde kalır.
- 1 sırayla 9, 8, 5, 3, 2 ile karşılaştırılır. Hepsi bir kaydırılır, 1 başa yerleşir: `1 2 3 5 8 9`

Tasarım:

- Yerleştirme işlemi 1. indeksten başlar ve $n-1$. indekse kadar sürer.
- Yerleştirilecek eleman geçici bir değişkene (`tmp`) alınır.
- Geriye doğru taşıma döngüsünün adım sayısı belli değildir. Eleman yerini bulunca durulur. Bu yüzden `while` kullanılır.
- En kötü durumda en son 1. ile 0. eleman karşılaştırılır. Bu yüzden şart $j > 0$'dır.

Akış diyagramı (ekrandan):

- $i = 1$'den $n-1$'e kadar:
  - `tmp = a[i], j = i`
  - `(j > 0) && (tmp < a[j-1])` olduğu sürece: `a[j] = a[j-1], j = j-1`
  - Döngüden çıkınca `a[j] = tmp`

Ara adım örneği (hocanın tahtadaki anlatımı): Baş tarafı sıralı bir dizide 3. gözdeki 4 yerleştirilir.

- `tmp = 4`, `j = 3`. 4, 2. gözdeki 7'den küçük. 7, 3. göze kaydırılır. `j = 2`.
- 4, 1. gözdeki 5'ten küçük. 5, 2. göze kaydırılır. `j = 1`.
- 4, 0. gözdeki elemandan küçük değil. Döngüden çıkılır. 4, 1. göze yerleştirilir.

C karşılığı (akış diyagramından ajan tarafından çevrildi, derlenip `3 2 8 5 9 1` dizisiyle test edildi):

```c
void insertion_sort(int a[], int n) {
    int i, j, tmp;
    for (i = 1; i <= n - 1; i++) {
        tmp = a[i];
        j = i;
        while (j > 0 && tmp < a[j - 1]) {
            a[j] = a[j - 1];
            j--;
        }
        a[j] = tmp;
    }
}
```

- En kötü durum: Dizi tersten sıralıysa her eleman başa kadar taşınır. Bunun bir çaresi yoktur.
- En iyi durum: Dizi sıralıysa her eleman bir kez karşılaştırılıp durur. Toplam $n-1$ karşılaştırma yapılır.

Shell Sort

Insertion Sort'un büyük dizilerdeki sorunu şudur: Başta olması gereken bir eleman sondaysa (örneğin 1000 elemanlı dizide 900. gözde), onu tek tek başa taşımak zaman alır. Shell Sort bu sorunu çözmek için geliştirilmiştir.

- Insertion Sort'a dayanır. Farkı, elemanı büyük adımlarla öne taşımasıdır.
- Bir adım büyüklüğü (step size) belirlenir. Her eleman, kendinden bir adım önceki gözdeki elemanla karşılaştırılır.
- Adım 3 ise dizi, her biri kendi içinde Insertion Sort ile sıralanan 3 alt diziye ayrılmış gibi davranır.
- Adım her turda ikiye bölünür. Adım 1 olunca son tur klasik Insertion Sort olur.

Örnek (ekrandan, 01:30:58): Hoca örnekteki 44'ü, başa kadar taşınışı görülsün diye 4 yaptı.

```
indeks:  0   1   2   3   4   5   6   7   8
dizi   : 54  26  93  17  77  31   4  55  20      step = 3

Alt diziler (3 atlayarak):
```

```
  0, 3, 6 -> 54 17  4   ->  4 17 54
  1, 4, 7 -> 26 77 55   -> 26 55 77
  2, 5, 8 -> 93 31 20   -> 20 31 93

step = 3 sonunda : 4  26  20  17  55  31  54  77  93
step = step/2 = 1: klasik Insertion Sort
Sonuç            : 4  17  20  26  31  54  55  77  93
```

- $i = 3$ ile başlanır, çünkü adım 3'tür. 17, 3 önceki gözdeki 54 ile karşılaştırılır. 54 kaydırılır, 17 öne geçer.
- 4, önce 54 ile, sonra 17 ile karşılaştırılır. Tek tek değil, 3'er atlayarak başa taşınır.
- 20 de aynı şekilde 93 ve 31'i atlayarak hızla öne gelir.

Öğrenci soruları:

- **Hız tam olarak adım sayısı kadar mı artar?** Tam olarak söylenemez. Ortalama duruma göre değişir. Ama Insertion Sort'tan daha hızlıdır.
- **Adım büyüklüğünü kim belirler?** Küçük dizilerle çalışırken başlangıç adımını programcı verir. Sonra ikiye bölünerek gidilir. Hoca genelde adımın $n/3$ civarında alındığını, çok büyük alınmaması gerektiğini söyledi.
- **Alt diziler neye göre sıralanır?** Her alt dizi kendi içinde Insertion Sort ile sıralanır.

Adım dizileri ve performans:

- Klasik Shell Sort'ta adımlar $n/2, n/4, \ldots, 1$ şeklindedir. 1000 elemanlı dizide ilk grup 1., 500., 999. elemanlardan oluşur.
- Knuth'un önerdiği adım dizisi 1, 4, 13, ... şeklindedir.
- Rastgele dizilmiş 100.000 sayı üzerinde yapılan bir testte Shell Sort'un Insertion Sort'tan 600 kat hızlı olduğu bulunmuştur.
- Shell Sort, Insertion Sort'tan en az iki kat hızlıdır. Karmaşıklığı yaklaşık $n^{3/2}$'dir.
- Yine de küçük dizilerde Insertion Sort, Shell Sort'a göre daha çok tercih edilir.

C karşılığı (derste Shell Sort'un kodu yazılmadı. Aşağıdaki kod ajan tarafından eklendi, derlenip yukarıdaki örnek diziyle test edildi):

```c
void shell_sort(int a[], int n, int step) {
    int i, j, tmp;
    for (; step >= 1; step /= 2)
        for (i = step; i < n; i++) {
            tmp = a[i];
            j = i;
            while (j >= step && tmp < a[j - step]) {
                a[j] = a[j - step];
                j -= step;
            }
            a[j] = tmp;
        }
}
```

Karmaşıklık Analizi

Bir algoritmanın karmaşıklığı, girişte n eleman varken işlemin n'e bağlı olarak kaç kez yapılacağıdır.

- "Şu işlemcide 10 dakika çalışıyor" bilgisi karşılaştırma için anlamlı değildir. Bunun için tüm şartların aynı olması gerekir.
- Bu yüzden herkes için ortak bir ölçü kullanılır: giriş boyutu n'e bağlı karmaşıklık.

Üç durum incelenir:

- **En iyi durum (best case):** En önemsiz durumdur. Pratikte karşılaşılması beklenmez. Örneğin her zaman sıralı bir dizi gelmez.
- **En kötü durum (worst case):** En çok dikkate alınan durumdur.
- **Ortalama durum (average case):** Sayıların ortalamasını almak değildir. Rastgele bir giriş için beklenen değerdir, yani istatistiksel bir kavramdır.

Arama örneği:

- En iyi durumda aranan eleman dizinin ilk gözündedir.
- En kötü durumda son gözündedir.
- Ortalama durumda herhangi bir gözde olabilir.

Bir öğrenci ortalama durumun daha önemli olduğunu, çünkü daha sık gerçekleştiğini söyledi. Hoca, pek çok çalışmada ortalama durumun daha önemli olduğunu onayladı.

Planlama örnekleri:

- **Deprem anında telefon hatlarının kilitlenmesi:** Operatörler planlamayı nadir görülen en kötü duruma göre yapmamıştır. Çünkü bu durum çok seyrek görülür ve maliyeti artırır.
- **Sınav sonuçları açıklanınca ÖSYM sitesine erişilememesi:** Bu durum yılda birkaç kez olsa da önemlidir ve önceden bilinir. Burada en kötü duruma göre planlama yapmak daha doğru olabilir.
- Sonuç: En kötü durum ortalama durumdan farklıysa ihtiyaç değerlendirilmeli. Hangisine göre önlem alınacağına tasarımcı karar verir.

Bubble Sort analizi:

- En iyi durum (dizi sıralı): $n-1$ karşılaştırma, yani n.
- En kötü durum (dizi ters sıralı): $i = 1$ için $n-1$, $i = 2$ için $n-2$, ..., son turda 1 karşılaştırma.

$$\sum_{i=1}^{n-1} i = \frac{(n-1)n}{2} = \frac{n^2-n}{2} \approx n^2$$

- n sonsuza giderken 2'ye bölmenin önemi yoktur. En kötü durum karmaşıklığı n^2'dir.

Selection Sort analizi:

- Önce $n-1$, sonra $n-2$, ..., en son 1 karşılaştırma yapılarak en küçük eleman bulunur.
- Toplam karşılaştırma $\frac{n(n-1)}{2}$, yani yaklaşık n^2.
- Dizinin sıralı olup olmadığına dair bilgi kullanılmaz. En iyi durum en kötü duruma eşittir: ikisi de n^2.

Insertion Sort analizi:

- En iyi durum (sıralı dizi): $n-1$ karşılaştırma, yani n.
- En kötü durum (ters sıralı, örneğin `5 4 3 2 1`): $i = 1$ için 1, $i = 2$ için 2, ..., $i = n-1$ için $n-1$ karşılaştırma.
- Toplam $\frac{n(n-1)}{2} = \frac{n^2-n}{2}$, yani n^2.
- Ortalama durumu yaklaşık $\frac{n^2}{4}$ mertebesindedir. Bu da Bubble Sort ve Selection Sort'tan daha iyidir. Ayrıntılı hesap Algoritma Analizi dersinde yapılıyor.

Shell Sort analizi: Karmaşıklığı adım dizisine (increment sequence) bağlıdır. En kötü durumda $n^{3/2}$'den başlayıp $n \log n$ ve $n \log^2 n$ mertebelerine kadar değişen sonuçlar vardır.

Neden n^2 algoritmalar büyük dizilerde kullanılmaz? Eleman sayısı büyüdükçe çalışma süresi çok uzar. Küçük dizilerde ise ortalama durumda en verimlisi Insertion Sort olduğu için en çok o tercih edilir.

Karmaşıklığın Çalışma Süresine Etkisi

Hocanın "çok sevdiği" tablo (ekrandan, 02:16:50):

n	$\lg n$	n	$n \lg n$	n^2	2^n	$n!$
10	0,003 µs	0,01 µs	0,033 µs	0,1 µs	1 µs	3,66 ms
10^8	0,027 µs	0,1 s	2,66 s	115,7 gün	4×10^{13} yıl	-

- $n = 10$ iken $n \log n$ ile n^2 arasında hissedilir bir fark yoktur.
- $n = 10^8$ olduğunda $n \log n$ algoritma 2,66 saniye sürer. n^2 algoritma ise 115,7 gün sürer.
- $n!$ için $n = 10^8$ hesabı yapılmamıştır. $n = 20$ iken bile 77,1 yıl sürer.
- Insertion, Selection ve Bubble Sort n^2; Quick Sort, Merge Sort ve Heap Sort $n \log n$ algoritmalardır.

Çıkarımlar:

- Tasarım yaparken ortamın şartları düşünülmelidir. Örneğin üniversitenin online sisteminde en kötü durum, yaklaşık 20 bin öğrencinin aynı anda bağlanmasıdır. Küçük bir iş yerinde ise 100 kişi olabilir.
- Derin öğrenme (deep learning) sistemlerinin eğitimi aylar sürebilir. Hocaya göre bu alanda optimizasyon çoğu zaman düşünülmüyor. Daha verimli tasarımla bu süreler kısaltılabilir.
- Bir öğrenci sürelerin CPU'da mı hesaplandığını sordu. Hoca, bunların eski kişisel bilgisayarlarda yapılmış hesaplar olduğunu söyledi. Önemli olan, aynı donanımda n ile n^2 arasındaki makasın nasıl açıldığını görmektir.

Yerinde Sıralama (In-place Sorting)

- Yedek dizi kullanmadan, dizinin kendi üzerinde yapılan sıralamadır.
- Selection Sort, Insertion Sort ve Quick Sort yerinde sıralama yapar.
- Merge Sort yerinde değildir. Sıralama için yedek dizi kullanır.
- Eleman sayısı büyükken yedek yer kullanmak iyi değildir. Bu yüzden genelde yerinde çözüm tercih edilir. Bu yalnız sıralamada değil, pek çok algoritmada geçerlidir.
- Yine de Merge Sort, $n \log n$ algoritmalar içinde tercih edilen bir algoritmadır:
  - Quick Sort'un en kötü durum karmaşıklığı n^2'dir.
  - Merge Sort'ta en iyi, en kötü ve ortalama durumların hepsi $n \log n$'dir.
  - Bu yüzden şartlara göre ek yer kullanımı tolere edilebilir.

Kararlı Sıralama (Stable Sorting)

İkinci bir ölçüte göre sıralama yapılırken, ilk sıralamadaki eşit değerlerin sırası bozulmuyorsa algoritma kararlıdır.

Örnek: Kişiler önce adlarına göre sıralanmış olsun. Sonra doğum yılına göre sıralansın.

Ada göre sıralı	Doğum yılı
Ali	1999
Ayşe	2001
Banu	1998
Berna	1999
Canan	2001

- Kararlı bir algoritmayla doğum yılına göre sıralanınca sonuç: Banu (1998), Ali (1999), Berna (1999), Ayşe (2001), Canan (2001).
- 1999 doğumlular arasında Ali, Berna'dan önceydi. Yeni sıralamada da önce kalır. Berna, Ali'nin üstüne çıkmaz.
- Excel'de öğrencileri önce notlarına, ikinci olarak numaralarına göre sıralamak buna örnektir.

Hangi algoritmalar kararlı?

- **Insertion Sort kararlıdır:** Eleman kendinden öncekilerle karşılaştırılır. Eşit elemanlar birbirinin önüne geçmez.
- **Bubble Sort kararlıdır:** Eşit olan ardışık elemanlar yer değiştirmez. Bir öğrenci, en sona gitmesi gereken bir elemanın tüm diziyi geçerken sırayı bozup bozmayacağını sordu. Hoca, bu elemanın eşit değerdeki bir elemanla yer değiştirmeyeceğini, bu yüzden kalan sıranın korunacağını açıkladı.
- **Selection Sort kararsızdır:** Eşitler arasında kıyas yapmaz. En küçük eleman hangisine denk geldiyse onunla yer değiştirir.

- **Quick Sort kararsızdır.**

Ders sonrası notlar

- Bubble Sort'un en kötü durum toplamında hoca önce toplamın üst sınırını $n-2$ olarak yazdı. Bir öğrenci toplamın 1'den $n-1$'e kadar olması gerektiğini söyledi. Hoca düzeltti. Doğrusu: $\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$.
- Bubble Sort'un döngü sınırlarını kontrol ettiği örnekte (`3 9 5 11 4`) hoca üçüncü turun sonucunu yanlış yazdı (`3 5 4 9 11` bıraktı). Bir öğrenci düzeltti. Doğru ara adımlar: 1. tur `3 5 9 4 11`, 2. tur `3 5 4 9 11`, 3. tur `3 4 5 9 11`.
- Tablodaki 2^n sütununda $n = 10^8$ satırına yazılan 4×10^{13} yıl değeri aslında $n = 100$ içindir (işlem başına 1 ns varsayımıyla 2^{100} ns $\approx 4 \times 10^{13}$ yıl). $n = 10^8$ için 2^n çok daha büyüktür.
- Hoca tabloyu anlatırken bazı değerleri farklı okudu ("0,07 mikrosaniye", "0,01 saniye"). Doğru değerler tahtadaki tablodaki gibidir: $\lg n$ için 0,027 µs, n için 0,1 s.
- Bubble Sort'un en iyi durumunun $n-1$ karşılaştırma olması yalnız erken çıkışlı (`swapped` kontrollü) sürüm için geçerlidir. İlk tasarımda dizi sıralı olsa da $\frac{n(n-1)}{2}$ karşılaştırma yapılır.
- Klasik Shell Sort adım dizisinin ($n/2, n/4, \ldots$) en kötü durumu n^2'dir. Knuth dizisi ($h = 3h+1$: 1, 4, 13, 40, ...) en kötü durumda $O(n^{3/2})$ verir. $O(n \log^2 n)$ sonucu Pratt dizisine aittir.
- Java'da ilkel tipler (`int[]` gibi) için `Arrays.sort` çift pivotlu Quick Sort kullanır ve küçük parçalarda Insertion Sort'a geçer. Nesne dizilerinde ise kararlı bir Merge Sort türevi (TimSort) kullanılır.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.